

Spectral Composition: A New Paradigm for Sample-Based Music Creation via Latent Diffusion

Lorenzo Moriondo
tunedconsulting@gmail.com
lorenzo@genefold.ai

tuned.org.uk
Genefold AI Ltd

August 14, 2026

Abstract

We introduce *Spectral Composition*, a novel method for sample-based music production that treats composition as iterative navigation of a compressed spectral latent manifold rather than direct waveform or symbolic editing. A frozen ArrowSpace graph Laplacian L_F and its associated energy-dispersion network λ^{ED} , estimated once from the producer’s entire sample library, define a low-dimensional feature-space manifold over which all generative and compositional operations are performed. The method proceeds in four stages: **(1) spectral encoding** (“dehydration”) of the library into a compact spectral representation $(L_F, U_q, \lambda^{\text{ED}})$; **(2) dense unconditional sampling** of a sound bank via a spectrally-conditioned 1-D diffusion transformer (DiT) operating on EnCodec audio latents; **(3) MIDI-conditioned materialisation** (“rehydration”) that pitch-shifts and re-timbres generated sounds according to a language-model-generated MIDI file; and **(4) recursive variant generation** by feeding outputs back through the model with successive MIDI sequences. The resulting family of variants, each sharing spectral lineage with the source library, forms a natural set of stems for multitrack mixing. Spectral Composition is built on the Latent Sound Diffusion (LSD) system [1], which is a sound-production fork of ArrowSpace Latent Diffusion with Spectral Chart Conditioning (ALD-SC) [2], and is exposed to the producer via the LSD-Studio desktop application [3].

Contents

1	Introduction	3
1.1	Contributions	3
1.2	Paper Organisation	3

2	Background	3
2.1	Latent Diffusion for Audio	3
2.2	ArrowSpace Graph Priors and Energy-Dispersion Networks	4
2.3	Entropic Time Scheduling and the Barontini Clock	4
3	Spectral Composition	4
3.1	Stage 1 — Spectral Encoding (Library Dehydration)	5
3.1.1	EnCodec feature extraction	5
3.1.2	ArrowSpace prior construction	5
3.1.3	Two-stage model training	5
3.2	Stage 2 — Dense Unconditional Sampling (Sound-Bank Generation)	5
3.3	Stage 3 — MIDI-Conditioned Materialisation (Rehydration)	6
3.3.1	Language-model MIDI generation	6
3.3.2	Pitch-shifted decoding (synthesize_midi)	6
3.3.3	Audio-conditioned variant generation	6
3.4	Stage 4 — Recursive Variant Generation	6
3.4.1	Convergence and heat-death stopping	7
3.5	Final Assembly — Multitrack Mixing	7
4	System Architecture	7
4.1	Frozen ArrowSpace Prior and Manifold Construction	7
4.2	1-D DiT Denoiser	7
4.3	Graph Decoder	7
4.4	Inference Contract (A / B / C)	8
4.5	LSD-Studio: Producer UX and Mix Bus	8
5	Implementation and Datasets	8
5.1	Software Stack	8
5.2	Sound Archive and Licensing	8
5.3	Training Details	8
6	Evaluation	9
6.1	Perceptual Coherence of Recursive Variants	9
6.2	Reconstruction Fidelity: Graph Decoder vs. Baseline	9
6.3	Ablation Studies	9
6.4	Limitations and Open Issues	9
7	Discussion	10
7.1	Spectral Composition as a Creative Paradigm	10
7.2	Relationship to Generative-AI Music Tools	10
7.3	Ethical and Licensing Considerations	10
8	Future Work	10
8.1	Text/CLAP Conditioning and Genre Control	10
8.2	Long-Form Generation and Wave Recurrence	10
8.3	Full Entropic Training Schedules	10
8.4	Federated and Personalised Priors	11
9	Conclusion	11

1 Introduction

Traditional sample-based music production treats a sample library as a fixed, enumerable set of audio assets: the producer browses, triggers, slices, and layers clips inside a digital audio workstation (DAW), guided by perception and taste. This workflow has scaled to industrial size through large commercial sample libraries, but it fundamentally limits compositional novelty to recombination of existing clips without access to the generative structure of the collection.

Spectral Composition proposes a different model: the sample library is a *corpus* from which a frozen spectral prior is estimated once, after which composition is an act of sampling and recombination inside the resulting latent manifold. This reframes “browsing samples” as “traversing a spectral graph”, and “arranging a track” as “recursively rehydrating dense latent draws against symbolic control signals.”

1.1 Contributions

- We define the Spectral Composition pipeline: a four-stage method (dehydration, sampling, materialisation, recursion) that operationalises compositional workflows over a spectral latent manifold.
- We show how the existing LSD inference contract (`generate_sound_bank`, `condition_on_audio`, `synthesize_midi`) directly implements stages 2–4 of the pipeline, requiring only the addition of a language-model MIDI front-end.
- We articulate the mathematical and architectural foundations of the method, grounding them in the ALD-SC research programme’s central claim: decoding on the feature-space manifold (L_F, λ^{ED}) yields better global semantic coherence under compression than decoding on an unconstrained ambient latent.
- We describe LSD-Studio as the producer-facing realisation of the pipeline, assigning multitrack mixing and workflow management to the application layer.

1.2 Paper Organisation

Section 2 reviews background in latent diffusion for audio, ArrowSpace graph priors, and entropic time scheduling. Section 3 defines the four stages of Spectral Composition. Section 4 describes the underlying model architecture. Section 5 details the implementation and datasets. Section 6 presents evaluation methodology. Section 7 discusses the method as a creative paradigm. Section 8 outlines future work, and Section 9 concludes.

2 Background

2.1 Latent Diffusion for Audio

Latent diffusion models [8] move the expensive iterative denoising process from pixel (or waveform) space into a compressed continuous latent space produced by a pretrained autoencoder, drastically reducing computation while retaining high-fidelity synthesis. EnCodec [9] provides the audio equivalent: a neural codec operating at 24 kHz that

encodes mono audio clips into a 1-D continuous latent $z \in \mathbb{R}^{D \times T}$ (with $D = 128$, $T = 375$ for 5-second clips) via its pre-quantization continuous features.

[Placeholder: Discuss related audio generative models (AudioLM, MusicGen, Stable Audio, AudioCraft). Contrast with symbolic-only (MIDI) and waveform-only generation. Motivate the spectral-latent approach.]

2.2 ArrowSpace Graph Priors and Energy-Dispersion Networks

ArrowSpace [6] defines a spectral search framework for embeddings in which feature columns of a corpus feature matrix $A \in \mathbb{R}^{N \times F}$ are treated as nodes of a feature-space graph $G_F = (V_F, W_F)$ with $|V_F| = F$. The feature-space graph Laplacian

$$L_F = D^{-1/2}(D - W)D^{-1/2}, \quad L_F = U\Lambda U^\top, \quad (1)$$

has eigenvectors $U_q \in \mathbb{R}^{F \times q}$ (the q smoothest modes) that define the semantic subspace of the corpus.

The Energy Dispersion Network (EDN) [7] associates to each feature f a per-feature energy-dispersion scalar $\lambda_f^{\text{ED}} \in [0, 1]$, encoding how semantic structure is distributed over the feature graph. Projected onto the chart, these give per-mode dispersion weights

$$\lambda_k^{\text{chart}} = \sum_{f=1}^F \lambda_f^{\text{ED}} U_{f,k}^2. \quad (2)$$

[Placeholder: Expand on the ArrowSpace spectral search paper, the kNN graph construction, τ -modulation, and the connection between λ^{ED} and semantic coherence under compression.]

2.3 Entropic Time Scheduling and the Barontini Clock

The entropic (Barontini) clock [10, 11] provides an intrinsic, spectrally-informed stopping criterion for diffusion sampling. Per-mode entropy-time coordinates are defined as

$$\tau_k(t) = \nu_k \bar{\alpha}_k(t), \quad (3)$$

where ν_k are the eigenvalues of L_F and $\bar{\alpha}_k(t)$ is the mode- k noise schedule. The sampler terminates when

$$\sum_{k=1}^q \nu_k \bar{\alpha}_k(t) < \varepsilon, \quad (4)$$

the “heat-death” criterion, replacing an arbitrary fixed step count with a spectrally-motivated halting rule [1, 2].

[Placeholder: Discuss the connection to the problem of time in quantum gravity and cold-atom experiments (Barontini 2026). Relate to prior work on adaptive diffusion schedules (DDIM, DPM-Solver, etc.).]

3 Spectral Composition

We now define the four stages of the Spectral Composition pipeline formally. Let $\mathcal{X} = \{x_1, \dots, x_N\}$ denote the producer’s sound archive (a set of raw audio clips), and let $\mathcal{M}$ denote a trained LSD model (frozen prior + graph decoder + 1-D DiT).

3.1 Stage 1 — Spectral Encoding (Library Dehydration)

3.1.1 EnCodec feature extraction

Each clip $x_i \in \mathcal{X}$ is encoded by the frozen EnCodec encoder E_ϕ (24 kHz mono, never updated) to produce:

$$(z_i, A_i) = E_\phi(x_i), \quad z_i \in \mathbb{R}^{D \times T}, \quad A_i = \text{Pool}(z_i) \in \mathbb{R}^F. \quad (5)$$

The pooled feature field A_i is the row contributed by clip i to the corpus feature matrix $A \in \mathbb{R}^{N \times F}$.

3.1.2 ArrowSpace prior construction

From A the ArrowSpace adapter builds the frozen feature-space graph Laplacian L_F and energy-dispersion network λ^{ED} via the `pyarrowspace` library [5]. The eigenvectors U_q and eigenvalues ν_k are stored as non-trainable buffers. The spectral conditioning vector for a clip is

$$c_{\text{spec}} = [\tilde{e}, \lambda^{\text{chart}}, \nu] \in \mathbb{R}^{3q}, \quad (6)$$

where $\tilde{e}_k$ is the per-mode band energy. The entire library is now represented by the compact frozen objects $(L_F, U_q, \lambda^{\text{ED}})$ — the *dehydrated* spectral representation.

3.1.3 Two-stage model training

Training proceeds as two frozen-then-fine-tuned phases. In Phase 1 the graph decoder is trained with optional latent-space noise injection (parameter `NOISE_INJECT` $\in [0, \infty)$). In Phase 2 the 1-D DiT denoiser is trained on the corpus latents $\{z_i\}$ conditioned on c_{spec} :

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{z_0, \epsilon, t} \|v - v_\theta(z_t, t, c_{\text{spec}})\|_2^2, \quad (7)$$

using v -prediction for numerical stability.

[Placeholder: Describe the full training loss including $\mathcal{L}_{\text{rec}}$ (L1 + multi-scale STFT), $\mathcal{L}_{\text{chart}}$, and $\mathcal{L}_{\text{smooth}}$. Discuss the non-repeatable training design (SEED=None, NOISE_INJECT).]

3.2 Stage 2 — Dense Unconditional Sampling (Sound-Bank Generation)

Given trained $\mathcal{M}$, a sound bank $\mathcal{B}$ of n clips is generated by running the DDIM or Euler sampler on the 1-D DiT unconditionally, then decoding through the `ClockGatedGraphDecoder`:

$$\hat{z}^{(j)} \sim p_\theta(z \mid c_{\text{spec}}), \quad j = 1, \dots, n, \quad (8)$$

$$\hat{x}^{(j)} = D_\psi(\hat{z}^{(j)}; U_q, \lambda^{\text{ED}}, c_{\text{spec}}^{(j)}), \quad (9)$$

where $c_{\text{spec}}^{(j)}$ is derived *self-consistently* from $\hat{z}^{(j)}$ itself (no separate chart sampling required). The bank $\mathcal{B} = \{\hat{x}^{(1)}, \dots, \hat{x}^{(n)}\}$ consists of samples that are perceptually self-similar — drawn from the same frozen manifold — yet acoustically distinct because `SEED=None` is the default.

[Placeholder: Discuss the “super-dense” character of the bank: why all samples share stylistic identity while varying locally. Relate to the frozen prior design constraint (corpus-level prior, not per-clip).]

3.3 Stage 3 — MIDI-Conditioned Materialisation (Rehydration)

3.3.1 Language-model MIDI generation

A language model (e.g. a GPT-based symbolic music model) generates a MIDI file $\mathcal{P} = \{(p_i, t_i, d_i)\}$ where p_i is the MIDI pitch, t_i the onset time, and d_i the duration of note i .

[Placeholder: Describe the GPT-MIDI generation front-end: prompt design, conditioning on genre/key/BPM, and the interface to LSD-Studio. Discuss alternative symbolic sources (human MIDI, rule-based generation, MusicXML).]

3.3.2 Pitch-shifted decoding (synthesize_midi)

For each note (p_i, t_i, d_i) the adapter selects a bank clip $\hat{x}^{(j)}$ and applies pitch-shifting to MIDI pitch p_i via the `synthesize_midi` method of `LSDModel`:

$$y_i = \text{PitchShift}(\hat{x}^{(j)}, \Delta p_i) \Big|_{[t_i, t_i+d_i]}, \quad (10)$$

where Δp_i is the semitone offset from the bank clip’s reference pitch. The materialised output is

$$Y^{(1)} = \sum_i y_i, \quad (11)$$

a waveform that follows the symbolic structure of $\mathcal{P}$ while carrying the spectral identity of the source library.

3.3.3 Audio-conditioned variant generation

Optionally, the materialised output $Y^{(1)}$ is fed back into the model via `condition_on_audio` (Mode B) to generate variants:

$$\hat{z}' \sim p_\theta(z \mid z_{\text{cond}} + \eta), \quad \eta \sim \mathcal{N}(0, \sigma^2 I), \quad (12)$$

where $z_{\text{cond}} = E_\phi(Y^{(1)})$ is the conditioning latent and σ is the perturbation strength. The chart conditioning c_{spec} is re-derived from $\hat{z}'$, keeping the output on the archive manifold.

3.4 Stage 4 — Recursive Variant Generation

Stages 2 and 3 are applied recursively. Let $\mathcal{P}^{(1)}, \dots, \mathcal{P}^{(R)}$ be R distinct MIDI files (generated by the language model or supplied by the producer). The recursive scheme is:

$$\mathcal{B}^{(0)} = \text{generate_sound_bank}(n), \quad (13)$$

$$Y^{(r)} = \text{synthesize_midi}(\mathcal{P}^{(r)}, \mathcal{B}^{(r-1)}), \quad (14)$$

$$\mathcal{B}^{(r)} = \text{condition_on_audio}(Y^{(r)}, n, \sigma_r), \quad r = 1, \dots, R. \quad (15)$$

Each round deepens the stylistic imprint of the source library while incorporating new symbolic content from $\mathcal{P}^{(r)}$. The family $\{Y^{(1)}, \dots, Y^{(R)}\}$ constitutes a set of *variants* related by shared spectral ancestry.

3.4.1 Convergence and heat-death stopping

The recursive process is bounded intrinsically by the heat-death criterion (Eq. 4): if the conditioning latent z_{cond} has collapsed to a near-constant spectral chart, additional conditioning passes yield diminishing novelty. In practice, the producer controls depth via R and the perturbation schedule $\{\sigma_r\}$.

3.5 Final Assembly — Multitrack Mixing

The variant outputs $\{Y^{(1)}, \dots, Y^{(R)}\}$ plus any selections from the unconditional bank $\mathcal{B}^{(0)}$ form a natural stem set for a multitrack mix. Within LSD-Studio, these are routed to a mix bus with per-stem gain, panning, and effect chains. Additional post-processing (overlap-and-add long-form generation, BPS-synchronised modulation, latent-space concatenation) is available via the notebook series (02–04) [1].

[Placeholder: Describe the DAW integration and MIDI synchronisation in LSD-Studio. Discuss how BPM and key are propagated from the LM-generated MIDI through to the mix.]

4 System Architecture

4.1 Frozen ArrowSpace Prior and Manifold Construction

The ArrowSpace prior is built once from the corpus by `build_prior.py` and stored as non-trainable buffers:

$$\Pi_q = U_q U_q^\top \in \mathbb{R}^{F \times F}, \quad (16)$$

the spectral projector onto the smooth subspace. The frozen design constraint means that L_F and U_q never receive gradient updates; only the graph decoder and DiT are trained. This ensures that the manifold geometry is a property of the *corpus*, not a function of any single training run.

[Placeholder: Describe the kNN graph construction, the ArrowSpace library API (`gl`, `lambdas()`), and the fallback behaviour when `pyarrowspace` bindings are unavailable.]

4.2 1-D DiT Denoiser

The denoiser `MinimalDiT` patchifies the 1-D latent $z \in \mathbb{R}^{D \times T}$ with stride- s convolutions, applies L transformer blocks with AdaLN conditioning on (t, c_{spec}) , and unpatchifies to predict velocity v . Classifier-free guidance (CFG) is applied by randomly dropping c_{spec} during training with probability p_{drop} , enabling guidance-scale control at inference.

4.3 Graph Decoder

The `ClockGatedGraphDecoder` processes the denoised latent $\hat{z}$ through a sequence of `WaveReconstructionBlocks`:

$$\hat{H} = A \cdot U_q, \quad (17)$$

$$g = \sigma(W_g \cdot c_{\text{spec}}) \odot \bar{\alpha}(t), \quad (18)$$

$$A' = (\hat{H} \odot g) \cdot U_q^\top + A_{\text{res}}, \quad (19)$$

where A_{res} is a residual convolution path and $\bar{\alpha}(t)$ is the Barontini clock gate (weak early in denoising, strong late). The decoder thus allocates reconstruction energy according to the corpus dispersion λ^{ED} , not arbitrary learned weights.

4.4 Inference Contract (A / B / C)

`LSDModel` (defined in `src/ald_sc/inference.py`) exposes three methods that directly implement Stages 2–4:

Mode A: `generate_sound_bank(n, steps, temperature, seed)` Unconditional DiT sampling + graph decode. Implements Stage 2.

Mode B: `condition_on_audio(audio, n, strength, seed)` Conditioned sampling with SDE bridge toward the input latent. Used in Stage 4 feedback loops.

Mode C: `synthesize_midi(midi_path, bank, seed)` Pitch-shifted bank decoding driven by a MIDI file. Implements Stage 3.

4.5 LSD-Studio: Producer UX and Mix Bus

LSD-Studio [3] is a Rust + `iced` desktop application that wraps the Python LSD model workers and exposes the full Spectral Composition workflow to the producer: library ingestion, model baking, sound-bank auditioning, MIDI sequencing, recursive generation, and multitrack mixing. The application builds to a single self-contained binary; Python workers are embedded at compile time.

[Placeholder: Architecture diagram of the IPC bridge between Rust frontend and Python workers. Describe the preset system and mix-bus signal flow.]

5 Implementation and Datasets

5.1 Software Stack

The LSD model core is implemented in Python 3.13 with PyTorch ≥ 2.2 , using `uv` for dependency management. The graph prior is computed via `pyarrowspace` (Rust bindings). LSD-Studio is implemented in Rust (stable) with the `iced` GUI framework and communicates with Python workers over a JSON-RPC IPC channel.

5.2 Sound Archive and Licensing

LSD-Studio ships with support for four commercially-licensed datasets, all carrying CC0 or CC BY 4.0 licences suitable for commercial production:

5.3 Training Details

Two design decisions make each training run artistically non-repeatable: **(i) SEED=None**: a fresh random seed is sampled at the start of each run, so no two trained models are identical; **(ii) NOISE_INJECT**: a scalar $\sigma_{\text{inj}} \geq 0$ controls noise injected into the EnCodec latent z before the graph decoder’s training pass, increasing acoustic variety at the cost of reconstruction fidelity.

Table 1: Supported sound archive datasets.

Dataset	Size	Content	Licence
NSynth [12]	300K notes	Musical notes, 1006 instruments	CC BY 4.0
Clotho [13]	~5K clips	Environmental audio with captions	CC BY 4.0
FSD50K-CC0	~10K clips	Diverse environmental audio	CC0
Freesound Loop Dataset	~9700 clips	Loop clips	CC0

[Placeholder: Table of hyperparameters (learning rate, batch size, decoder/DiT architecture dimensions, q , K , $NOISE_INJECT$ sweep). Training time on target hardware.]

6 Evaluation

6.1 Perceptual Coherence of Recursive Variants

[Placeholder: Propose a listening study evaluating whether recursive variants are perceived as stylistically related to the source library across R rounds. Metrics: pairwise timbral similarity (e.g. CLAP embedding cosine distance), spectral centroid/rolloff drift over rounds, producer preference ratings.]

6.2 Reconstruction Fidelity: Graph Decoder vs. Baseline

The primary controlled comparison isolates the graph decoder (**WaveReconstructionBlock** + U_q + λ^{ED}) against a baseline decoder of identical channel widths and upsampling strides but with plain **ResBlock1d** (no U_q , no λ^{ED}).

[Placeholder: Report multi-scale STFT reconstruction loss, audio FAD (once dependency issues with `fadtk` are resolved), and per-band spectral energy retention for graph vs. baseline on ESC-50 and NSynth subsets.]

6.3 Ablation Studies

[Placeholder: Ablation over: (1) q (number of retained spectral modes), (2) $NOISE_INJECT$ level, (3) CFG guidance scale, (4) heat-death threshold ε , (5) recursion depth R . Metrics: diversity (mean pairwise distance in CLAP space) vs. fidelity (STFT loss).]

6.4 Limitations and Open Issues

[Placeholder: Discuss: FAD computation (`fadtk` removed; multi-scale STFT used as fallback); real-data experiments pending (ESC-50, NSynth); pitch-shifting artefacts in `synthesize_midi`; scalability of ArrowSpace graph construction to large archives ($N > 10K$); and the absence of long-form ($> 8s$) compositional structure in the current recursive scheme.]

7 Discussion

7.1 Spectral Composition as a Creative Paradigm

[Placeholder: Contrast Spectral Composition with (a) loop-based composition (static library, manual arrangement), (b) sample-based AI tools that generate independent clips (AudioCraft, Stable Audio), and (c) purely symbolic generative models (MusicGen, Jukebox). Argue that the manifold-constrained, recursively rehydrated workflow occupies a distinct position: it maintains strong stylistic identity with the producer’s own archive while enabling genuine novelty.]

7.2 Relationship to Generative-AI Music Tools

[Placeholder: Survey of relevant systems: Google MusicLM, Meta AudioCraft/MusicGen, Stability Audio, Riffusion, Moûsai, AudioLDM 2. Situate Spectral Composition relative to each with respect to: (1) source identity preservation, (2) producer control, (3) symbolic grounding via MIDI, (4) recursive/iterative generation.]

7.3 Ethical and Licensing Considerations

[Placeholder: Discuss: training on commercially-licensed datasets (all CC0/CC BY 4.0); the dehydration step as a form of model-mediated compression of the producer’s private library; implications for sample clearance; and the “sound of the future” framing as a creative rather than extractive application.]

8 Future Work

8.1 Text/CLAP Conditioning and Genre Control

[Placeholder: Extend Stage 2 and Stage 3 with text and CLAP conditioning (tracked as LSD Phase 2). Discuss joint text + graph CFG guidance and genre-conditioned bank generation.]

8.2 Long-Form Generation and Wave Recurrence

[Placeholder: Describe the overlap-and-add notebook (02) and latent-space concatenation (04) as current approximations to long-form composition. Motivate the ESDM wave-recurrence mechanism [4] as a principled replacement, enabling second-order graph-wave propagation in the decoder.]

8.3 Full Entropic Training Schedules

[Placeholder: The current model uses the entropic clock only at inference (sampling stopping criterion). Discuss training with per-mode noise schedules as an open research direction [2] (issue #11).]

8.4 Federated and Personalised Priors

[Placeholder: Motivate federated training of ArrowSpace priors across multiple producers' private archives without sharing raw audio, as a privacy-preserving extension of the dehydration step.]

9 Conclusion

Spectral Composition introduces a four-stage compositional workflow — dehydration, dense sampling, MIDI-conditioned materialisation, and recursive variant generation — that grounds sample-based music production in a compressed spectral latent manifold estimated from the producer's own library. The method is fully implemented in the LSD model core and LSD-Studio application, requiring only the addition of a language-model MIDI front-end to complete the pipeline. By anchoring every generative and compositional operation to the same frozen ArrowSpace prior $(L_F, U_q, \lambda^{\text{ED}})$, Spectral Composition ensures that all generated and rehydrated sounds share spectral lineage with the source archive, yielding variant families that are simultaneously novel and stylistically coherent.

[Placeholder: Summarise the main experimental findings (to be filled once evaluation is complete).]

References

- [1] tuned-org-uk. Latent Sound Diffusion (LSD): Sound-generation fork of ALD-SC. GitHub repository, 2026. <https://github.com/tuned-org-uk/latent-sound-diffusion>
- [2] tuned-org-uk. ArrowSpace Latent Diffusion with Spectral Chart Conditioning (ALD-SC). GitHub repository, 2026. <https://github.com/tuned-org-uk/arrow-space-latent-diffusion>
- [3] tuned-org-uk. LSD Studio: Latent Sound Diffusion desktop application. GitHub repository, 2026. <https://github.com/tuned-org-uk/LSD-studio>
- [4] tuned-org-uk. Entropic Semantic Diffusion Model (ESDM). GitHub repository, 2026. <https://github.com/tuned-org-uk/entropic-semantic-diffusion>
- [5] tuned-org-uk. pyarrow-space: ArrowSpace library (Rust bindings). GitHub repository, 2026. <https://github.com/tuned-org-uk/pyarrow-space>
- [6] L. Moriondo et al. ArrowSpace: Spectral search for embeddings. *Journal of Open Source Software*, 2026. <https://doi.org/10.21105/joss.09002>
- [7] tuned-org-uk. Energy Dispersion Networks. arXiv:2606.21535, 2025. <https://arxiv.org/abs/2606.21535>
- [8] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.
- [9] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi. EnCodec: High fidelity neural audio compression. arXiv:2210.13438, 2022.

- [10] F. Barontini. Testing the problem of time with cold atoms. *Physical Review Letters*, 2026.
- [11] I. Stancevic et al. Entropic time schedulers for generative diffusion models. arXiv preprint, 2025.
- [12] J. Engel et al. Neural audio synthesis of musical notes with WaveNet autoencoders. In *ICML*, 2017.
- [13] K. Drossos, S. Lipping, and T. Virtanen. Clotho: An audio captioning dataset. In *ICASSP*, 2020.
- [14] K. J. Piczak. ESC: Dataset for environmental sound classification. In *ACM Multimedia*, 2015.
- [15] tuned.org.uk. Diffusion as spectral-geometric projection, 2024. https://www.tuned.org.uk/posts/021_diffusion_as_spectral_geometric_projection/